

First-Class Values

In most languages, variables hold data (like strings or integers), and functions do things to that data. But what if a function itself could be treated exactly like a piece of data?

In Go, functions are **First-Class Values**. This means anything you can do with an integer, you can also do with a function:

1. You can save a function inside a variable.
2. You can pass a function as an argument to another function.
3. You can return a function as a result.
4. You can store multiple functions inside a Slice or a Map.

1. Assigning a Function to a Variable

You can declare a variable and **set its type to be a specific function signature**.

- **The Signature:** This is the exact blueprint of the function (what it takes in, and what it returns).

```
// A function that takes an integer and returns an integer.  
var increment func(int) int
```

- **The Assignment:** You can assign an existing function to this variable.

```
myVar := increment
```

Note: Notice there are no parentheses () after **incrementFunction**! If you added parentheses, you would be executing the function and saving its integer result. By leaving them off, you are saving the function itself in the variable. You can now execute it later by calling: **myVar(5)**.

2. Passing Functions as Arguments (Callbacks)

Because a function is just data, you can pass it into another function. The receiving function can then execute it whenever it wants.

```
func applyIt(anyMathFunc func(int) int, num int) {  
    anyMathFunc(num)  
}  
  
applyIt(increment, 5)  
applyIt(decrement, 5)
```

- The **applyIt** function doesn't know what math it is doing. It just knows it received a function and the number 5, and it blindly applies the function to the number. You could pass it an **increment** function, a **decrement** function, or a **multiply** function, making **applyIt** incredibly flexible!

3. Anonymous Functions (Lambdas)

If you are only going to pass a small, simple function as an argument once, it is a waste of space to formally declare and name it at the top of your file. Instead, you can define it on the fly, directly inside the function call.

- **What it is:** A function with no name.

```
applyIt(func(x int) int { return x + 1 }, 5)
```

Instead of passing a named variable, you literally write the function logic directly inside the argument slot.

Expert Takeaway

Why is this concept so important for backend architecture? **Middleware!** When you build an API in Go, an incoming HTTP request often needs to pass through several checks before it reaches your database. For example:

1. Log the request time.
2. Check if the user's JWT token is valid.
3. Finally, execute the actual database logic.

In Go, we handle this using First-Class Functions. We write a generic **Authenticator** function and pass our database function as an argument. The **Authenticator** checks the **token**, and if it's valid, it calls the **database** function we handed to it. This creates a beautifully modular, highly secure "**chain**" of logic!